1 theorem

Algorithm 1: The Asymptotic PTAS for 1D VRP with MDTW

Input: instance $I = \{G, Q, W\}$
Output: the solution S of I

```
Rounding: demand rounding and location rounding.
Partitioning: partitioning at depots and between depots.
Solving: solving middle instances, side instances.
Touring: creating the solution S for I.
```

Theorem 1. *Alg.1 has the time complexity* $\mathrm{O}(n \log n^2)$ *and the approximation ratio* $1 + \mathrm{O}(\epsilon)$*, under the asymptotic condition* $r_{max} = OPT/(1/\epsilon)^{\mathrm{o}(1/\epsilon)}$.

2 lemma

Algorithm 2: Position Rounding

Input: instance $I = \{G, Q, W\}$
Output: position-rounded instance I'

```
set l_0 = ε r_max / n;
for each pair of adjacent depots (d_l, d_r) do
    for i ∈ ℕ_0 s.t. between (d_l, d_r) do
        place the i-th rounded location at l_0(1+ε)^i to d_l.
        place the i-th rounded location at l_0(1+ε)^i to d_r.
    for each customer node c between (d_l, d_r) do
        move c to its nearest rounded location.
for the leftmost and rightmost depots (d_lm, d_rm) do
    for i ∈ ℕ_0 s.t. (1+ε)^i ≤ r_max do
        place the i-th rounded location at (1+ε)^i from left to d_lm.
        place the i-th rounded location at (1+ε)^i from right to d_rm.
    for each customer node c between (d_l, d_r) do
        move c to its nearest rounded location.
```

Lemma 1. *(Time Complexity and Approximation Ratio of Rounding Step) Given an instance* I *with problem scale* n *of 1D VRP with MDTW, Alg. 2 and 3 output the rounded instance* I''*. Denoting the optimum solution of* I'' *by* opt'' *and its total tour length by* OPT''*, then* opt'' *can be modified to a feasible solution* S *of* I*. We have*

Algorithm 3: Demand Rounding

Input: position-rounded instance I'
Output: demand-rounded instance I''

```
for each rounded position p do
    for for each time window tw do
        if n_p^{tw} ≥ 1/ε^2 then
            sort the big demands in Q_p^{tw}.
            partition Q_p^{tw} into {Q_{p,i}^{tw}}, i ∈ [1, 1/ε^2].
            for each Q_{p,i}^{tw} do
                get the biggest demand q̄ ∈ Q_{p,i}^{tw}.
                for each q ∈ Q_{p,i}^{tw} do
                    round q to q̄.
```

(1) The time complexity of Alg. 2 and 3 is $\mathrm{O}(\mathrm{Poly}(n))$.
(2) The time complexity of modifying opt'' *to* S *is* $\mathrm{O}(\mathrm{Poly}(n))$.
(3) S *has the total tour length* $(1+\mathrm{O}(\epsilon))OPT''$.
(4) $OPT'' = (1+\mathrm{O}(\epsilon))OPT$.

Algorithm 4: Partitioning at Depots

Input: rounded instance I''
Output: sub-instance sequence $(I_i)_{0\le i\le m}$

```
sort D in the order from left to right into sequence (d_i)_{1≤i≤m}.
create I_0 whose node set is {d_1} ∪ C_0.
create I_i, 1 ≤ i ≤ m − 1, whose node set is {d_i, d_{i+1}} ∪ C_i
create I_m whose node set is {d_m, C_m}.
```

Lemma 2. *Given an rounded instance* I'' *of problem scale* n *of 1D VRP with MDTW, Alg. 4 partitions it at depots into sub-intances* $(I_i)_{0\le i\le m}$. *Denoting the optimum solution of any instance of 1D VRP with MDTW by* $opt(\cdot)$, *and its total tour length by* $OPT(\cdot)$, *then* $\bigcup_{0\le i\le m}\{opt(I_i)\}$ *is a feasible solution* S'' *of* I''. *We have*

(1) The time complexity of Alg. 4 is $\mathrm{O}(\mathrm{Poly}(n))$.
(2) $\sum_{0\le i\le m} OPT(I_i) = OPT(I'')$.

Lemma 3. *For each* $I_i, 1 \le i \le m-1$, *there exists* j_1^* *in* $[1, 1/\epsilon]$ *ways of outer-partitioning such that it partitions* I_i *into the optimum outer-instances* (I_l^*, I_b^*, I_r^*), *which satisfy*

$$OPT(I_l^*) + OPT(I_b^*) + OPT(I_r^*) \le (1+O(\epsilon))OPT(I_i).$$

Lemma 4. *For each I_{d}, there exists j_2^* in $[1, 1/\epsilon]$ ways of inner-partitioning such that it partitions I_{d} into a set of the optimum regions $\{R_k^*\}$, which satisfy*

$$\sum_k OPT(R_k^*) \leq (1 + O(\epsilon))OPT(I_{\mathrm{d}}).$$

Algorithm 5: Middle instance solving algorithm

Input: middle-instace I_{b}
Output: tour set S_{b}

```
relax I_b to I_R.
solve LP and obtain its solution (x*_f)_{f∈F}.
round (x*_f)_{f∈F} up to (x̄_f)_{f∈F} = ⌈x*_f⌉_{f∈F}.
construct initial S_b according to (x̄_f)_{f∈F}.
for each customer (p_i, q_i, tw) in the decreasing order of r_i do
    for each customer (p_i, q_i, tw) in the increasing order of tw do
        if ∃ t ∈ S_b s.t. c_t ≥ q_i and t^tw passes i then
            use t to serve i.
            set c_t = c_t − q_i.
        else
            find the nearest depot d_t to p_i.
            creat m_t whose tw-th segment is {d_t, p_i, flag} and other segments
              are ∅.
            set d_t = { the left depot, if flag = r.
                      { the right depot, if flag = l.
            add a new tour t = {d_t, {(p_i, q_i, tw)}, m_t} in S_b to serve i.
            set c_t = Q − q_i.
```

Lemma 5. *Alg. 5 outputs its solution S_{b} in time complexity $Poly(n)$. It computes the solution $(x_f^*)_{f\in F}$ for LP, which is rounded up to $(\bar{x}_f)_{f\in F} = \lceil x_f^* \rceil_{f\in F}$, and subsequently be constructed to solution S_R for the input I_{b}. We have*

(1) $Length(S_R) < OPT(I_b) + (1/\epsilon)^{O(1/\epsilon)} r_{max}$.
(2) $Length(S_{\mathrm{b}}) \leq (1 + 2w\epsilon)Length(S_R) + 4r_{max}$.

Lemma 6. *Alg. 1 has time complexity $Poly(n)$, and its solution S_d has $Length(S_d) < OPT(R_{farthest}) + \sum_{k\in\mathbb{N}_0} OPT(R_k) + (1/\epsilon)^{O(1/\epsilon)} \cdot r_d$, here r_d is maximum radius in I_d.*

3 Overall Analysis of Asymptotic PTAS

This section presents the overall performance analysis of the Alg. 1 for one-dimensional vehicle routing problems, that is, it proves that the algorithm is an asymptotic polynomial-time approximation scheme.

3.1 For time complextity

It is easy to see that the time complexity of the algorithm depends on the solving stage. According to lemma16 and lemma18, the time complexity of the intermediate instance solving algorithm5 and the two-sided instance solving algorithm1 is 1 and $\mathrm{O}(n^2 \log n)$ respectively. Therefore, the time complexity of solving phase is $(m+1) \cdot (1/\epsilon)^2 \cdot \mathrm{O}(n^2 \log n) = \mathrm{O}(n^2 \log n)$, the theorem is proved.

3.2 For approximation ratio

Analyze each phase of the asymptotic polynomial-time approximation scheme in turn. Firstly, according to the lemma**??**, the rounded solution to the problem can be modified to the solution of the original problem with a total path length of $(1+\mathrm{O}(\epsilon))OPT$ by restoring the position and need of each client. Then, according to lemma2, the union of the optimal solution of all sub-instances between two sites is the optimal solution of the original problem. According to lemma3 and lemma4, there exists optimal first-layer and second-layer split method, so that all the paths given by the intermediate instance solving algorithm and the two-sided instance solving algorithm constitute the solution of the original problem, whose total length is $(1+\mathrm{O}(\epsilon))OPT$. Finally, according to lemma17 and lemma19, the solving phase can give solution of the original problem with the total path length $(1+\mathrm{O}(\epsilon))OPT + (1/\epsilon)^{\mathrm{O}(1/\epsilon)} \cdot r_{max}$, then under the asymptotic condition $r_{max} = OPT/(1/\epsilon)^{\mathrm{O}(1/\epsilon)}$, the total path length of the solution given by the asymptotic polynomial-time approximation scheme is $(1+\mathrm{O}(\epsilon))OPT$, the theorem is proved.

4 Performance Analysis of Rounding Step

This section presents the properties of the position rounding algorithm 2-2 and the demand rounding algorithm 2-3 during the rounding phase through Lemma 2.3. For the ease of following the proof, the rounding step is illustrated in Fig. 1. This lemma allows for the optimal solution of the rounded instance to be found first and then modified into a feasible solution for the original problem, with the guarantee that the total path length of this feasible solution will not exceed $1+O(\varepsilon)$ times that of the optimal solution of the original problem. First, Lemma 2.1 and 2.2 are introduced, which will be used to support the proof of Lemma 2.3 Lemma 2.1 (Lower Bound Lemma for Client Radii):

Lemma 7. *For any instance I of the one-dimensional vehicle routing problem, the following property holds:*

$$Rad(I) = \frac{2}{Q} \sum_{i \in C} q_i \cdot r_i \leq OPT. \tag{1}$$

Here, q_i represents the cost associated with the i-th request, and $r_i = \min_{e \in d} dist(c_i, d)$ is the radial distance.

rounding.pdf

Fig. 1. Position rounding and demand rounding for left depot

Lemma 8. *For any instance I of the one-dimensional vehicle routing problem, any feasible route t can be truncated into w segments. Furthermore, assuming that the leftmost and rightmost customer nodes served by t during the time window tw are located at pl and pr, respectively, and the segment for the time window $tw-1$ ends at position p_{end}^{tw-1}, then the segment for the time window tw satisfies the following condition:*

1. *If $p_l^{tw} \leq p_r^{tw} \leq p_{end}^{tw-1}$, then the segment in time window tw begins with p_l^{tw} and ends with p_r^{tw}.*
2. *If $p_{end}^{tw-1} \leq p_l^{tw} \leq p_r^{tw}$, then the segment in time window tw begins with p_{end}^{tw-1} and ends with p_r^{tw}.*
3. *If $p_l^{tw} \leq p_{end}^{tw-1} \leq p_r^{tw}$, then the segment in time window tw begins with p_l^{tw} and ends with p_{end}^{tw-1}, and the following sub-conditions hold:*
   - *From p_{end}^{tw-1} to p_l^{tw}, and from p_l^{tw} to p_r^{tw}, the vehicle travels directly;*
   - *From p_{end}^{tw-1} to p_r^{tw}, and from p_r^{tw} to p_l^{tw}, the vehicle travels directly.*

Proof. When the first case occurs, any feasible segment of t in the time window tw must cover the interval $[p_{\text{end}}^{tw-1}, p_l^{tw}]$. Among them, the straight segment from p_{end}^{tw-1} to p_l^{tw} is the shortest, thus the first case is established. Similarly, the second case also holds. When the third case occurs, any feasible segment of t in the time window tw must cover the interval $[p_l^{tw}, p_r^{tw}]$. Therefore, the first straight segment of this part must proceed from position p_{end}^{tw-1} to p_l^{tw} or p_r^{tw}. Among these, the polyline segment $(p_{\text{end}}^{tw-1}, p_l^{tw}, p_r^{tw})$ or $(p_{\text{end}}^{tw-1}, p_r^{tw}, p_l^{tw})$ is the shortest, so the third case holds, and the lemma is proven.

Corollary 1. *For any instance I of the one-dimensional vehicle routing problem, the length of each path type f can be computed in constant time complexity $O(w) = O(1)$.*

Lemma 9. *For a sub-instance I between two stations of the one-dimensional vehicle routing problem, assume that the instance I after the rounding phase is processed is denoted as I', and the total length of the optimal solution opt' for I' is OPT'. Then, the following two properties can be concluded:*

1. *opt' can be modified to be a feasible solution for I with a total length of $(1 + O(\epsilon))OPT'$;*
2. *$OPT' = (1 + O(\epsilon))OPT$.*

Proof. First, we prove property (1). Consider lines 1-5 of the position rounding algorithm 2-2, which is the rounding process for customer positions between each pair of adjacent stations. Select any path t from opt', and define t_{tw} as its segment on the tw-th time window. For each segment t_w, if the total length of the path is shortened after modifying opt' into a feasible solution for I, then property (1) obviously holds. If the total length of the path increases, then assume the ending position of the segment of t on the time window $tw - 1$ is p_{end}, and the leftmost and rightmost positions of the customers served during the time window tw are p_l and p_r respectively. Then, by Lemma 6, it is known that the increase in length is necessarily due to the leftward restoration of p_l or the rightward restoration of p_r, and without loss of generality, it suffices to analyze p_r.

When p_r is rounded from its original position to the left to some insertion position $l_0 \cdot (1+\epsilon)^x$ coming from the left station, because the original position of p_r is between two adjacent insertion positions, and the distance between these two insertion positions is:

$$l_0 \cdot (1 + \epsilon)^{x+1} - l_0 \cdot (1 + \epsilon)^x = l_0 \cdot (1 + \epsilon)^x \cdot \epsilon = \epsilon \cdot p_r \leq \epsilon \text{Length}(t), \tag{2}$$

Therefore, it can be concluded that the additional length on tw is at most $\epsilon \cdot \text{Length}(t)$; similarly, when p_r is rounded to some insertion position coming from the right station, by considering these insertion positions from right to left, the distance between the insertion positions on either side of the original position is still $l_0 \cdot (1 + \epsilon)^{x+1} - l_0 \cdot (1 + \epsilon)^x \leq \epsilon \cdot \text{Length}(t)$, thereby concluding that the additional length on tw is at most $\epsilon \cdot \text{Length}(t)$. Since there are w time windows, the modified length of opt' is at most

$$\sum_{t \in \text{opt}'} (1 + w \cdot \epsilon)\text{Length}(t) = (1 + O(\epsilon))\text{OPT}'. \tag{3}$$

Then, consider lines 6-8 of the position rounding algorithm 2-2, which pertain to the rounding process of customer positions located to the left of the leftmost station and to the right of the rightmost station in path t of opt'. Without loss of generality, we need only analyze the latter case (as shown in Figure 2-6). By Lemma 2.2, it is known that the increase in length of the segment t^{tw} of t is necessarily due to the original position of p_l being rounded to the right to the insertion position coming from the rightmost station $(1 + \epsilon)^i$. Because the original position of p_l is between two adjacent insertion positions, and the distance between these two insertion positions is

$$(1+\epsilon)^i - (1+\epsilon)^{i-1} = (1+\epsilon)^{i-1} \cdot \epsilon < (1+\epsilon)^i \cdot \epsilon = \epsilon \cdot p_r \leq \epsilon \text{Length}(t), \quad (4)$$

Therefore, it can be concluded that the increased length on t_w is at most $\epsilon \cdot \text{Length}(t)$, and considering there are w time windows, the modified length of opt' after the adjustments is at most

$$\sum_{t \in \text{opt}'} (1 + w \cdot \epsilon)\text{Length}(t) = (1 + O(\epsilon))\text{OPT}'. \quad (5)$$

In summary, property (1) can be concluded, namely that opt' can be modified to be a feasible solution for instance I with a total length of $(1 + O(\epsilon))\text{OPT}'$.

Next, we prove property (2). For the instance processed by the position rounding, a new instance $\underline{I}$ is defined in the reverse manner of the demand rounding algorithm 2-3, that is, rounding down each group of large demands to the largest demand in the adjacent lower group. Let the optimal solution for $\underline{I}$ be $\text{opt}(\underline{I})$, with its total path length being $\text{OPT}(\underline{I})$. Since the demand of each customer is reduced, it is obvious that

$$\text{OPT}(\underline{I}) \leq \text{OPT}. \quad (6)$$

Moreover, apart from the $n_p^{tw}\epsilon^2$ largest demands, $\text{opt}(\underline{I})$ can serve all other large demands at location p generated by the demand rounding algorithm 2-3 within the time window tw. Therefore, if a path is added for each of these unserved customers, and the total path length of the feasible solution composed of these paths together with $\text{opt}(\underline{I})$ is denoted as $\text{OPT}(\overline{I})$, then it can be obtained that

$$\text{OPT}' \leq \text{OPT}(\overline{I}) \leq \text{OPT} + \sum_{tw} \sum_{p} \left(n_p^{tw} \epsilon^2 \cdot 2r_p \right). \quad (7)$$

Since removing the time window constraints would reduce the total length of the optimal solution, then, based on the Lower Bound Lemma for Client Radii 2.1 and the definition of large demands, it is known

$$\frac{2}{Q} \sum_{tw} \sum_{p} n_p^{tw} (\epsilon \text{Q} \cdot r_p) \leq \frac{2}{Q} \sum_{tw} \sum_{p} n_p^{tw} (q_p^{tw} \cdot r_p) \leq \text{OPT}. \quad (8)$$

Furthermore, there is the expression

$$\sum_{tw} \sum_{p} \left(n_p^{tw} \epsilon^2 \cdot 2r_p \right) \leq \epsilon \text{OPT}. \quad (9)$$

Therefore, the property (2) is obtained as follows, and the proof of this lemma is completed.

$$\text{OPT}' \leq (1 + O(\epsilon))\text{OPT}. \quad (10)$$

5 Performance analysis of Partitioning Step

For the partitioning phase, this section presents the properties of the partitioning algorithm 2-4 at the stations and the inter-station partitioning algorithm 2-5 respectively, through Lemma 2.4, Lemma 2.7, and Lemma 2.8. First, Lemma 2.4 allows for the subsequent independent solution of each sub-instance I_i between two stations, whose optimal solutions can be combined into the optimal solution for the rounded instance. For a clear panarama of the regions and blocks in inner partitioning, we illustrated them in Fig. 2.

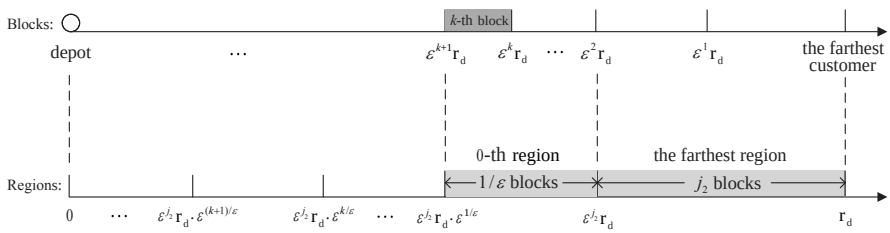

Fig. 2. The inner partitioning includs regions and blocks.

Lemma 10. *For an instance I of the one-dimensional vehicle routing problem, the time complexity of the partitioning algorithm 2-4 at the stations that divides I into a sequence of sub-instances between two stations $(I_i)_{0\le i\le m}$ is polynomial in n. Let the total path length of the optimal solution opt(I_i) for the sub-instance I_i be denoted as $OPT(I_i)$, then we have the expression*

$$\sum_{0\le i\le m} OPT(I_i) = OPT(I). \tag{11}$$

Proof. It is easy to see that the running time of the station-based rounding algorithm 2-4 depends on the sorting process. Therefore, any sorting algorithm with a time complexity of $O(n\log n)$ can be used, from which it follows that the running time of the algorithm is $O(n\log n)$.

For the total path length of the optimal solution set $\sum_{0\le i\le m}\mathrm{opt}(I_i)$, consider any path t in the optimal solution opt of I. Suppose t serves $x>1$ sub-instances $I_{t_1}, I_{t_2}, \ldots, I_{t_x}$ partitioned by algorithm 2-4, then t can be replaced by x paths $t_1, t_2, \ldots, t_x$, where t_i is the part of t in the sub-instance I_i^t. Clearly, the sum of the lengths of all paths t_i equals the length of path t, and thus we obtain $\sum_{0\le i\le m}\mathrm{OPT}(I_i) = \mathrm{OPT}(I)$.

Before presenting Lemma 2.7 and Lemma 2.8, two properties that support the lemma are introduced — the Long Path Property and the Micro Path Property. For the sub-instance I_i between two stations, and its first level of division $j_1 \in$

$[1, 1/\epsilon]$, the long path of I_i is defined as one of the following two types of paths: one that starts from the left station d_l and serves at least one customer node within the location interval $[(1-\epsilon^{j_1})L, L]$, or one that starts from the right station d_r and serves at least one customer node within the location interval $[0, \epsilon^{j_1}L]$.

Lemma 11 (Long Path Property). *For the sub-instance I_i between two stations and its first level of division $j_1 \in [1, 1/\epsilon]$, any long path t of I_i can be replaced by paths t_{lm} and t_r, which are not long paths, and the sum of their lengths satisfies Length(t_{lm}) + Length$(t_r) \leq (1 + 2w\epsilon^{j_1})$Length$(t)$.*

Proof. Consider the internal instances I_l, I_m, I_r generated by I_i under the first layer of division j_1, corresponding to the location intervals $[0, \epsilon^{j_1}L]$, $(\epsilon^{j_1}L, (1-\epsilon^{j_1})L)$, and $[(1-\epsilon^{j_1})L, L]$, respectively. Without loss of generality, let us assume that the long path t originates from the left station d_l of I_i. Then, replace it with two paths t_{lm} and t_r, where t_{lm} starts from d_l and serves all customers of t in the interval $[0, \epsilon^{j_1}L] \cup (\epsilon^{j_1}L, (1-\epsilon^{j_1})L)$, and t_r starts from d_r and serves all customers of t in the interval $[(1-\epsilon^{j_1})L, L]$. By the definition of a long path, t serves at least one customer node in the interval $[(1-\epsilon^{j_1})L, L]$, therefore $\text{Length}(t) \geq 2(1-\epsilon^{j_1})L$, and subsequently,

$$\text{Length}(t_r) \leq w \cdot 2\epsilon^{j_1}L \leq \frac{w\epsilon^{j_1}}{1-\epsilon^{j_1}} \cdot \text{Length}(t). \tag{12}$$

Then, when $\epsilon j_1 \leq 1/2$, the expression is

$$\frac{w\epsilon^{j_1}}{1-\epsilon^{j_1}} \leq 2w\epsilon^{j_1}. \tag{13}$$

Finally, it is clear that $\text{Length}(t_{lm}) \leq \text{Length}(t)$, which leads to the following result, thus completing the proof of the lemma.

$$\text{Length}(t_{lm}) + \text{Length}(t_r) \leq (1 + 2w\epsilon^{j_1})\text{Length}(t). \tag{14}$$

Denote the leftmost and rightmost customer positions served by a feasible path t for the sub-instance I_i between two stations as p_l and p_r, respectively. If they satisfy $(p_r/p_l) \leq 1/\epsilon$, then t is called a micro path. Here, since only sub-instances between two stations need to be solved, p_l and p_r are necessarily on the same side of either d_l or d_r. Below, the properties of micro paths are presented in Lemma 2.6.

Lemma 12 (Micro Path Property). *Any feasible path t for the sub-instance I_i between two stations can be replaced by a set of micro paths $T = \{t_i\}_{i>0}$, which satisfies Length$(T) \leq (1 + 2\epsilon)$Length(t).*

Proof. Denote the furthest customer position served by path t as p, and construct the set of micro paths T to replace it as follows: divide t into a series of path segments $(\epsilon^{i+1}p, \epsilon^i p]_{i\geq 0}$, then connect the start and end positions $\epsilon^{i+1}p$ of the i-th segment to the originating station of t, thus forming the i-th micro path t_i.

In the definition of the one-dimensional vehicle routing problem, customer nodes with radius $r_i \leq \epsilon r_{\max}/n$ are already excluded, so the number of micro paths here is finite, and the total number of paths in all micro path sets is less than n. Next, analyze the total length of micro paths in T. Clearly, t_i is a feasible path for I_i, and its length is $\leq 2(\epsilon^{i+1}p) + \text{part}_i$, where part_i is the length of path t in the location interval $(\epsilon^{i+1}p, \epsilon^i p]$. Therefore, the following result can be obtained.

$$\begin{aligned} \text{Length}(T) \leq \sum_{i\geq 0} \text{Length}(t_i) &= \sum_{i\geq 0} \left(2 \cdot \epsilon^{i+1}p + \text{part}_i\right) \\ &< \text{Length}(t) \cdot \left(\epsilon^1 + \ldots + \epsilon^\infty\right) + \text{Length}(t) \\ &= \text{Length}(t) \cdot \frac{\epsilon}{1-\epsilon} + \text{Length}(t). \end{aligned} \tag{15}$$

When $\epsilon \leq 1/2$, we have $1/(1-\epsilon) \leq 2$, which leads to the following result, thus completing the proof of the lemma.

$$\text{Length}(T) < \text{Length}(t) \cdot 2\epsilon + \text{Length}(t) = (1+2\epsilon)\text{Length}(t) \tag{16}$$

Then, based on the properties of the first and second level division methods provided by Lemmas 2.7 and 2.8, it is possible to solve only for the optimal solutions on each region given by these two division methods for I_i, and the sum of the path lengths of these optimal solutions will not exceed $(1 + O(\epsilon))$ times the optimal solution of I_i.

Lemma 13. *In all the first level division methods $j_1 \in [1, 1/\epsilon]$ for the sub-instance I_i between two stations, there exists an optimal division method j_1^* that decomposes I_i into internal instances I_l^*, I_b^*, I_r^*, which satisfy:*

$$OPT(I_l^*) + OPT(I_b^*) + OPT(I_r^*) \leq (1 + O(\epsilon))OPT(I_i). \tag{17}$$

The three terms on the left-hand side of the above inequality are the total path lengths of the optimal solutions opt(I_l^*), opt(I_b^*), and opt(I_r^*) for I_l^*, I_b^*, and I_r^*, respectively.

Proof. Consider the left region $R_l = [0, \epsilon^{j_1}L]$, the middle region $R_b = \{\epsilon^{j_1}L_i \leq (1-\epsilon^{j_1})L_i\}$, and the right region $R_r = [(1-\epsilon^{j_1})L_i, L_i]$ partitioned by j_1 on I_i, as well as their corresponding internal instances I_l, I_b, I_r. Consequently, define $\text{OPT}_{j_1} = \text{OPT}(I_l) + \text{OPT}(I_b) + \text{OPT}(I_r)$, where the terms on the right are the total path lengths of the optimal solutions $\text{opt}(I_l)$, $\text{opt}(I_b)$, and $\text{opt}(I_r)$ for I_l, I_b, and I_r, respectively.

By Lemmas 2.5 and 2.6, it is known that we can find a feasible solution T for the instance I_i that does not contain long paths, consisting only of micro paths, and its total length satisfies $\text{Length}(T) \leq (1 + O(\epsilon))\text{OPT}(I_i)$. Since T does not include long paths, customers in the left region R_l can only be served by paths originating from d_l, and customers in the right region R_r can only be served by paths originating from d_r.

Now, define T_{j_1} as the set of paths in T that serve customers in both regions R_l and R_r, and without loss of generality, we only need to analyze any path t originating from d_l. Truncate t into two paths t_l and t_b, where t_l and t_r serve the customer nodes of t in regions R_l and R_r respectively, and denote by T' the feasible solution of I_i after the aforementioned replacement. Clearly, $\text{Length}(t_l)+\text{Length}(t_b) \leq 2\text{Length}(t)$, and subsequently,

$$\text{OPT}_{j_1} \leq \text{Length}(T') \leq \text{Length}(T) + \sum_{t \in T_{j_1}} \text{Length}(t). \tag{18}$$

Now, we prove that for any other first-level division method i_1, the set of paths T_{i_1} that serve customers in the region $[0, \epsilon^{i_1} L_i] \cup (\epsilon^{i_1} L_i, (1-\epsilon^{i_1})L_i)$ and T_{j_1} do not share common paths, i.e., they are disjoint. When $i_1 > j_1$, since a path t serves customer nodes in R_b, its furthest served customer position must be greater than $\epsilon^{j_1} L_i$. On the other hand, as t is a micro path, its nearest served customer position must be greater than $\epsilon^{j_1+1} L_i \geq \epsilon^{i_1} L_i$. Thus, t does not serve any customer nodes in $[0, \epsilon^{i_1} L_i]$, i.e., t is not in T_{i_1}. When $i_1 < j_1$, since a path t serves customer nodes in R_l, its nearest served customer position must be less than or equal to $\epsilon^{j_1} L_i$. On the other hand, also because t is a micro path, its furthest served customer position must be less than or equal to $\epsilon^{j_1-1} L_i \leq \epsilon^{i_1} L_i$. Therefore, t does not serve any customer nodes in $(\epsilon^{i_1} L \leq (1-\epsilon^{i_1})L_i)$, i.e., t is not in T_{i_1}. In conclusion, T_{i_1} and T_{j_1} are disjoint, and thus we have

$$\sum_{j_1 \in [1,1/\epsilon]} \sum_{t \in T_{j_1}} \text{Length}(t) \leq \text{Length}(T_{j_1}) \leq \text{Length}(T). \tag{19}$$

Then, summing up all the first-level division methods, we can get

$$\begin{aligned} \sum_{j_1 \in [1,1/\epsilon]} \text{OPT}_{j_1} \leq \sum_{j_1 \in [1,1/\epsilon]} \left(\text{Length}(T') + \sum_{t \in T_{j_1}} \text{Length}(t) \right) \\ \leq (1/\epsilon + 1)\text{Length}(T). \end{aligned} \tag{20}$$

Finally, since there are $1/\epsilon$ terms for OPT_{j_1} on the left side of the above expression, there must exist an optimal first-level division method j_1^* satisfying $\text{OPT}(I_l^*) + \text{OPT}(I_b^*) + \text{OPT}(I_r^*) \leq \frac{1/\epsilon+1}{1/\epsilon} \cdot \epsilon \cdot \text{Length}(T) = (1+\epsilon)\text{Length}(T) \leq (1+O(\epsilon))\text{Length}(I_i)$, thus completing the proof of the lemma.

Next, the properties of the second-level division method j_2 are presented. First, let Id represent the instance I_0, I_m, or the internal instances I_l, I_r generated by the first-level division method j_1 for the sub-instance between two stations I_i. Then, denote the set of regions generated by j_2 as $R = \{R_i\}_{i=\{-,0,1,2,\ldots\}}$. Among these, R_- is the region farthest from the originating station $(\epsilon^{j_2} r_{\max}, r_{\max}]$, with the total path length of the optimal solution $\text{opt}(R_-)$ being $\text{OPT}(R_-)$; the $i \geq 0$-th region $(\epsilon^{j_2} r_{\max} \epsilon^{(i+1)/\epsilon}, \epsilon^{j_2} r_{\max} \epsilon^{i/\epsilon}]$ has the total path length of the optimal solution $\text{opt}(R_i)$ as $\text{OPT}(R_i)$. The theorem below states that there exists an optimal second-level division method $j_2^* \in [0, 1/\epsilon]$, such that the divided

$Ri^* \in R^*$ satisfies $\sum_{R_i^* \in R^*} \mathrm{OPT}(R_i^*) \leq (1+O(\epsilon))\mathrm{OPT}(I_d)$. Therefore, the overall property of the inter-station division algorithm is obtained, that is, for each instance I_i between two stations, there exists an optimal pair of division methods j_1^*, j_2^*, such that the total path length of the optimal solutions of the divided instances does not exceed $(1+O(\epsilon))\mathrm{OPT}(I_i)$. Consequently, the sum of the optimal solution lengths of all instances generated by the station-based and inter-station division algorithms will not exceed $(1+O(\epsilon))$ times the length of the optimal solution of the rounded instance, denoted by OPT'.

Lemma 14. *For the sub-instance I_i between two stations generated by the first-level division method j_1, and instance I_d, there exists an optimal second-level division method j_2^*, such that the divided region $R_i^* \in R^*$ satisfies*

$$\sum_{Ri^* \in R^*} OPT(R_i^*) \leq (1+O(\epsilon))OPT(I_d). \tag{21}$$

Proof. By Lemma 2.6, it is known that a feasible solution T_d containing only micro paths can be found for the instance Id, and its total length satisfies $\mathrm{Length}(T_d) \leq (1+O(\epsilon))\mathrm{OPT}(I_d)$. Since each region $Ri \in R$ divided by j_2 has the ratio of positions at its ends greater than or equal to $1/\epsilon$, and the ratio of the farthest to the nearest customer positions served by each micro path t is $p_r/p_l \leq 1/\epsilon$, each t in T_d spans at most two regions in Id. Here, let $\mathrm{OPT}_{j_2} = \sum_{R_i \in R} \mathrm{OPT}(Ri)$, and denote T_{j_2} as the set of paths in T_d that span two regions on Id. If each path t in T_{j_2} is truncated into two sub-paths that each serve the two regions spanned by t, then the expression is

$$\mathrm{OPT}_{j_2} \leq \mathrm{Length}(T_d) + \sum_{t \in T_{j_2}} \mathrm{Length}(t). \tag{22}$$

For any other second-level division method i_2, denote T_{i_2} as the set of paths in T_d that span two regions divided by i_2. If a path t in T_d is in T_{i_2}, then the two blocks it spans (located at the junction of two regions) in the second-level division method j_2 will necessarily belong to the same region. Therefore, T_{i_2} and T_{j_2} will not have any common paths, i.e., they are disjoint, and thus we have

$$\sum_{j_2 \in [1,1/\epsilon]} \sum_{t \in T_{j_2}} \mathrm{Length}(t) \leq \mathrm{Length}(T_{j_2}) \leq \mathrm{Length}(T_d). \tag{23}$$

Furthermore, summing up all the second-level division methods, we can get

$$\begin{aligned} \sum_{j_2 \in [1,1/\epsilon]} \mathrm{OPT}_{j_2} \leq \sum_{j_2 \in [1,1/\epsilon]} \left(\mathrm{Length}(T_d) + \sum_{t \in T_{j_2}} \mathrm{Length}(t) \right) \\ \leq (1/\epsilon + 1)\mathrm{Length}(T_d). \end{aligned} \tag{24}$$

Finally, since the left-hand side of the above expression contains $1/\epsilon$ terms of OPT_{j_2}, there must necessarily exist some optimal second-level division method j_2^* that satisfies $\sum_{R_i^* \in R^*} \mathrm{OPT}(R_i^*) \leq \frac{1/\epsilon+1}{1/\epsilon} \cdot \mathrm{Length}(T_d) = (1+\epsilon)\mathrm{Length}(T_d) \leq (1+O(\epsilon))\mathrm{OPT}(I_d)$, thus completing the proof of the lemma.

6 Performance Analysis of Solving Step

For the solving phase, in this section, the time complexity and approximation ratio of intermediate instance solving algorithm5 and two-sided instance solving algorithm1 are given by lemma16, lemma17 and lemma19, lemma18, these lemmas limit the time complexity of polynomial time approximation schemes to a polynomial of the size of the problem$n$ with approximation ratio$1 + O(\epsilon)$ of the calculation result. Firstly, lemma 15 that supports lemma16 are given. The most important componet of the solving step is the LP, so we illustrate its constraint in Fig. 3.

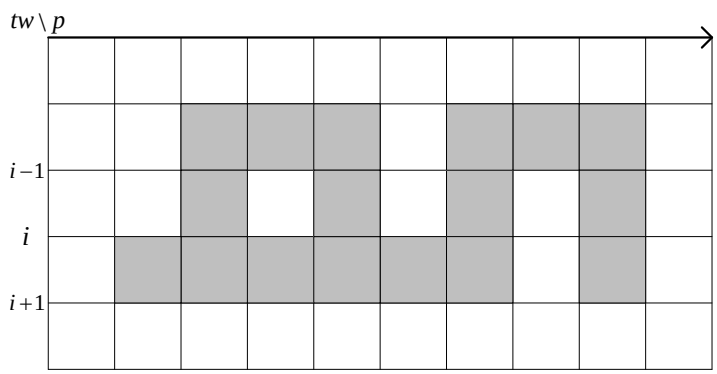

Fig. 3. The polygon in constraint (2-7) of linear programming

Lemma 15. *The length of all feasible paths between subinstances I_m of any two sites is no more than $4w \cdot r_{max}$.*

Proof. Firstly, supposing that the segment t^i of the feasible path t of I_m on the time window $tw = i$ satisfies $Length(t^i) > 4r_{max}$. Without loss of generality, suppose that path t starts from the left site of I_m, and the far left and far right location of client it served on the time window $tw = i$ are p_l, p_r respectively. Then, constructing two segments t' and t'' to replace t^i as fallows: t starts from the left site, reaches its end position on the time window $i - 1$, and connects to t', t' returns to p_l after it reaches r_{max}; t'' starts from the right site, serves all clients located at the right of r_{max} on t^i, and then returns to the right site, thus forming a new path. It is known from the lemma**??** that segment t' can connected to the part on the time window $tw \geq 2$ of the path t.

Next, proving that the above hypothesis is not valid. Obviously, $Length(t') \leq 2r_{max}$. For client nodes at the right of r_{max} on t^i, their radii can only correspond to right sites, thus there distance to right sites is less than r_{max}. Hence, $Length(t'') \leq 2r_{max}$, and then the sum of length of t' and t'' is less than or equal

to $4r_{max}$, which means the hypothesis $Length(t^i) > 4r_{max}$ is not valid. Therefore, the segment t^i of t on the time window $tw = i$ satisfies $Length(t^i) \leq 4r_{max}$, the length of t $Length(t) \leq 4r_{max}$, the lemma is proved.

The intermediate instance solving algorithm5 first relaxes the input intermediate instance I_b into each instance I_R whose small demand can be split, then solves linear programming (**??**) corresponding to I_R, rounding up its optimal solution can construct a feasible solution of all mass demand S_b in service I_b. Let $OPT(I_b)$ be the total path length of the optimal solution $opt(I_b)$ of I_b, lemma16 gives the time complexity and approximation ratio of S_b to $OPT(I_b)$ for solving linear programming, the approximation ratio of S_b after updating by greedy algorithm is given by lemma17.

Lemma 16. *The intermediate instance solving algorithm5 gives the optimal solution of linear programming $(x_f^*)_{f\in F}$ within constant time complexity, and the total length of the path set V corresponding to upward rounded $(\overline{x}_f)_{f\in F} = \lceil x_f^* \rceil_{f\in F}$ satisfies $Length(V) < OPT(I_b) + (1/\epsilon)^{O(1/\epsilon)} r_{max}$.*

Proof. As the time complexity of solving a linear programming is a polynomial of the number of variables and the number of constraints, the number of rounding positions, the number of requirement types and the number of feasible paths contained in them are analyzed step by step, so as to obtain the time complexity of the linear programming and the approximation ratio of V.

Number of Rounding Positions. After the rounding phase and split phase, every client in the intermediate instance I_b is at the location between $(\epsilon^j L, (1-\epsilon^j)L)$ with distance $(\epsilon r_{max}/n)(1+\epsilon)^i$ to left or right site, i is a integer, j is a first-level division mode, L is the distance between left site and right site. Let p_l, p_r represent the far left and the far right client position from the rounding position(the superscript is noted as i_l, i_r) starting from the left site in I_b and respectively, then

$$p_l = (\epsilon r_{max}/n)(1+\epsilon)^{i_l} > \epsilon^j L, \tag{25}$$

$$p_r = (\epsilon r_{max}/n)(1+\epsilon)^{i_r} < (1-\epsilon^j)L. \tag{26}$$

It follows that the number of rounding positions of client nodes in I_b is at most

$$\begin{aligned} 2(i_r - i_l) &\leq 2\left(\log_{(1+\epsilon)} \frac{(1-\epsilon^j)L}{\epsilon r_{max}/n} - \log_{(1+\epsilon)} \frac{\epsilon^j L}{\epsilon r_{max}/n}\right) \\ &\leq 2\log_{(1+\epsilon)} \frac{1-\epsilon^j}{\epsilon^j} < 2\log_{(1+\epsilon)} (\frac{1}{\epsilon})^j = \frac{2j \cdot \ln\frac{1}{\epsilon}}{\ln(1+\epsilon)}. \end{aligned} \tag{27}$$

Since $j \leq 1/\epsilon$, and obviously $1+\epsilon > 2^\epsilon$ when $0 < \epsilon < 1/2$, so

$$2(i_r - i_l) < \frac{2j \cdot \ln\frac{1}{\epsilon}}{\epsilon} \leq 2(\frac{1}{\epsilon})^2 \ln\frac{1}{\epsilon}. \tag{28}$$

Number of Requirement Types. After the rounding phase and split phase, there are at most $1/\epsilon^2 + 1$ mass demand on every time window tw at every rounding position, therefore, the number of requirement types in I_b is at most

$$2(i_r - i_l) \cdot w \cdot (1/\epsilon^2 + 1) = 2w(1/\epsilon)^2 \ln(1/\epsilon)(1 + 1/\epsilon^2). \tag{29}$$

Number of Feasible Paths. Since every path type (d_f, Y_f, m_f) can serve at most $1/\epsilon$ mass demand, the possible cases of Y_f can be obtained is at most $\sum_{1\le k\le 1/\epsilon}(2w(1/\epsilon)^2 \ln(1/\epsilon)(1+1/\epsilon^2))^k$. And the possible cases of m_f is at most $2w(2(1/\epsilon)^2 \log(1/\epsilon))^2$ according to its definition. Then, as every path can start from the left site or the right site, the number of path types is at most

$$2 \cdot \sum_{1\le k\le 1/\epsilon} (2w(1/\epsilon)^2 \log(1/\epsilon)(1 + 1/\epsilon^2))^k \cdot 2w(2(1/\epsilon)^2 \log(1/\epsilon))^2 = (1/\epsilon)^{\mathrm{O}(1/\epsilon)}. \tag{30}$$

Above all, the number of variables of linear programming is $(1/\epsilon)^{\mathrm{O}(1/\epsilon)}$, and the number of inequalities for the constraint (**??**) is $\mathrm{O}(1/\epsilon)^5$. Obviously, the number of inequalities for the constraint (**??**) is only related to the number of time windows w and the number of rounding positions $(1/\epsilon)^2 \ln(1/\epsilon)$, thus the linear programming can be solved in constant time complexity. In addition, since relaxation instance I_R must have an optimal solution, and the linear programming and I_R are one-to-one correspondence, the linear programming must also have an optimal solution. Therefore, when there are redundancies in the constraint (**??**), they will be eliminated in the simplex algorithm without affecting the solution of the optimal solution.

Next, analyzing the approximation ratio of V. Firstly, since every small demand in the linear programming are relaxed as divisible, so $\sum_{f\in F} Length(f)x_f^* \le OPT(I_b)$. Then, as the number of decision variables in the linear programming is at most $(1/\epsilon)^{\mathrm{O}(1/\epsilon)}$, thus there are at most $(1/\epsilon)^{\mathrm{O}(1/\epsilon)}$ fractions in the optimal solution x_f^*. According to the lemma15, the total length of the path in V is as fallows, the lemma is proved.

$$\begin{aligned} Length(V) &= \sum_{f\in F} Length(f)\overline{x}_f < \sum_{f\in F} Length(f)(x_f^* + 1) \\ &\le (\sum_{f\in F} Length(f)x_f^*) + (1/\epsilon)^{O(1/\epsilon)} \cdot Length(f) \\ &\le OPT(I_b) + (1/\epsilon)^{\mathrm{O}(1/\epsilon)} \cdot 4w \cdot r_{max} \end{aligned} \tag{31}$$

Lemma 17. *The solution of the intermediate instance solving algorithm5 S_b and the the path set V corresponding to upward rounded solution of the linear programming satisfy $Length(S_b) \le (1 + 2w\epsilon)Length(V) + 4r_{max}$.*

Proof. Suppose that any path added by the greedy algorithm served client nodes on the time window tw when created, then the path is equal to its segment on the time window tw. Thus, G^{tw} can be used to represent the set of all paths

newly added by greedy algorithm on the time window tw. Next, define sequence $(h_i^{tw})_{1\le i\le g}$, which is the ascending order of the radius of the client node served by the newly added paths in G^{tw} at the time of creation, and $h_0^{tw}=0$. And then let G_i^{tw} be the set of newly added paths whose radius of clients served satisfy $\ge h_i^{tw}$ when created, $G_i=\cup_{tw}G_i^{tw}$. It It should be noted that on each time window tw, the upper limit g of subscript of h_i^{tw} is set to the maximum value in all time windows, then as there is no new path in h_i^{tw}, just set $G_i^{tw}=0$, that is, the alignment of h_i^{tw} between different time Windows is completed. Therefore, the total length of paths in S_b is

$$Length(S_b)=Length(V)+2\sum_{1\le i\le g}(h_i-h_{i-1})|G_i|. \tag{32}$$

Next, let sequence $(p_x^{tw})_{1\le x\le g}$ represent the left to right arrangement of the client position served by the path in G^{tw} at the time of creation, and let $l_i^{tw}=\min_x\{p_x^{tw}\ \ge\ h_i^{tw}\}$ and $r_i^{tw}=\max_x\{L-p_x^{tw}\ \ge\ h_i^{tw}\}$ represent those closest positions that are at least h_i^{tw} from the left and right sites, respectively. Then, let V_i^{tw} be the set of paths in V serving the small demand in $small^{tw}(i)$, $V_i=\cup_{tw}V_i^{tw}$, $small^{tw}(i)$ represents the sum of all small demands in position interval $[l_i^{tw},r_i^{tw}]$ and on the time window tw.

$$small^{tw}(i)=\sum_{\substack{(p,q,tw)\in s:\\ l_i^{tw}\le p\le r_i^{tw}}}q. \tag{33}$$

Note $cap^{tw}(i)=\sum_{t\in V_i^{tw}}e_t$ as the remaining capacity of paths in V_i^{tw} before serving any small demand, where e_t represent the loaded capacity of t minus the mass demand it serves. Similarly, $cap(i)=\sum_{t\in V_i}e_t$. Next, considering iteration (i,tw) of greedy algorithm, now adding a new path to l_i^{tw} or r_i^{tw} if and only if there is no such path in S_b- it passes through a segment on the time window tw and has enough capacity to serve some small demand in $small^{tw}(i)$, then the remaining capacity of each path in V_i^{tw} is less than ϵQ. Therefore, the small demands in $small^{tw}(i)$ have been assigned to V_i^{tw} by the greedy algorithm satisfy $\ge cap^{tw}(i)-|V_i^{tw}|\epsilon Q+z_i^{tw+}-z_i^{tw-}$, where z_i^{tw+} represent small demands in $small^{tw}(i)$ served by other paths, z_i^{tw-} represent small demands on other time windows in $[l_i^{tw},r_i^{tw}]$ served by V_i^{tw}. Thus, the number of small demands in $small^{tw}(i)$ have not been in service is at most

$$small^{tw}(i)-cap^{tw}(i)+|V_i^{tw}|\epsilon Q-z_i^{tw+}+z_i^{tw-}. \tag{34}$$

For any pair of time windows a,b, any path $t1$ in V_i^a and any path $t2$ in V_i^b, the part of small demands in $small^a(i)$ served by $t2$ in z_i^{b-} iq equal to the corresponding part in z_i^{a+}, meanwhile, the part of small demands in $small^b(i)$ served by $t1$ in z_i^{a-} iq equal to the corresponding part in z_i^{b+}. Therefore, when summing over all time windows tw, z_i^{tw+}, z_i^{tw-} all cancel out. And then since h_i^{tw} is aligned between different time windows and the constraint(**??**) of the linear programming, the number of small demands in $[l_i=l_i^{tw},r_i=r_i^{tw}]$ have not been

in service in all time windows is at most

$$\sum_{tw} small^{tw}(i) - cap(i) + w|V_i^{tw}|\epsilon Q \leq w|V_i|\epsilon Q. \tag{35}$$

To sum up, limit the number of paths in G_i to
$|G_i| \leq \frac{w|V_i|\epsilon Q}{(1-\epsilon)Q} + 2 = \frac{w|V_i|\epsilon}{1-\epsilon} + 2$

$$\begin{aligned} Length(S_b) &= Length(V) + 2\sum_{1\leq i\leq g}(h_i - h_{i-1})|G_i| \\ &\leq Length(V) + \frac{w\epsilon}{1-\epsilon}\sum_{1\leq i\leq g} 2(h_i - h_{i-1})|V_i| + 4h_g \\ &\leq Length(V) + \frac{w\epsilon}{1-\epsilon}Length(V) + 4r_{max}. \end{aligned} \tag{36}$$

Since $\epsilon/(1-\epsilon) \leq 2\epsilon$ is true when $\epsilon \leq 1/2$, so the total length of paths in S_b satisfies $Length(S_b) \leq (1+2w\epsilon)Length(V) + 4r_{max}$. Furthermore, according to lemma5, $Length(S_b) < (1+\mathrm{O}(\epsilon))OPT(I_b) + (1/\epsilon)^{\mathrm{O}(1/\epsilon)} \cdot r_{max}$. The lemma is proved.

The two-sided instance solving algorithm1 takes instance I_0, I_m or the two-sided instance I_l, I_r of I_i as input I_d, and relaxes all small demands in I_d to be detachable, forming relaxation instance I_R. Then, invoke algorithm5 solve every areas divides by second-layer split methodj_2, finally combine paths in all areas as the final solution S_d. Let $OPT(I_d)$ be the total length of paths in the optimal solution $opt(I_d)$ of I_d, lemma18 and lemma19 give the time complexity and approximation ratio of the algorithm respectively.

Lemma 18. *The time complexity of the side-instance solving algorithm f1 is* $\mathrm{O}(n^2 \log n)$.

Proof. Firstly, analyzing the number of areas divided by the second-layer split method j_2. Since every client radius in the input instance I_d is greater than $\epsilon r_{max}/n > \epsilon r_d/n$, according to the definition of block, the number of blocks given by j_2 ia at most $\log n/\log(1/\epsilon)$. Further according by the definition of area, the area farthest from the originating site contains $j_2 \in [1, 1/\epsilon]$ blocks, and each of the other areas contains $1/\epsilon$ blocks, thus, there are total $O(\epsilon \log n/\log(1/\epsilon))$ areas to be solved.

Next, analyzing the number of rounding positions in every area. Since the area farthest from the originating site is $(\epsilon^{j_2} r_d, r_d]$, other areas of the ith$(i \geq 0)$ is $(\epsilon^{j_2} r_d \epsilon^{(i+1)/\epsilon}, \epsilon^{j_2} r_d \epsilon^{i/\epsilon}]$, if note the right position of an area as p, its left position $> \epsilon^{1/\epsilon} p$. Furthermore, the far left and the far right rounding position of this area is $(1+\epsilon)^{i_l}$, $(1+\epsilon)^{i_r}$ respectively, then

$$(1+\epsilon)^{i_l} > \epsilon^{1/\epsilon} p, \tag{37}$$

$$(1+\epsilon)^{i_r} \leq p. \tag{38}$$

Then, the number of rounding positions in every area is

$$\begin{aligned}(i_r - i_l) &< \log_{(1+\epsilon)} p - \log_{(1+\epsilon)}(\epsilon^{1/\epsilon} p) \\ &= \log_{(1+\epsilon)}(1/\epsilon)^{(1/\epsilon)} = (1/\epsilon)\log_{(1+\epsilon)}(1/\epsilon) = \frac{\frac{1}{\epsilon}\cdot \ln\frac{1}{\epsilon}}{\ln(1+\epsilon)}. \end{aligned} \tag{39}$$

When $0 < \epsilon < 1/2$, obviously, $1 + \epsilon > 2^{\epsilon}$, so

$$(i_r - i_l) < (\frac{1}{\epsilon})^2 \ln\frac{1}{\epsilon}. \tag{40}$$

Finally, adopt the same analysis method as the prove of lemma16, the number of path types in every areas is at most $(1/\epsilon)^{\mathrm{O}(1/\epsilon)}$, that is, the number of variables of the linear programming of the two-sided instance solving algorithm1 is $(1/\epsilon)^{O(1/\epsilon)}$, thus, this linear programming can be solved in constant time complexity. Obviously, the time complexity of adopting greedy algorithm to update the path set given by linear programming is $\mathrm{O}(n^2)$, therefore, the time complexity of the two-sided instance solving algorithm1 is $\mathrm{O}(\epsilon \log n/\log(1/\epsilon)) \cdot \mathrm{O}(n^2) = \mathrm{O}(n^2 \log n)$.

Lemma 19. *Given the second-layer split method j_2, the total length of paths in solution S_d solved by the two-sided instance solving algorithm1 is $Length(S_d) < \sum_i OPT(R_i) + (1/\epsilon)^{O(1/\epsilon)} \cdot r_d$, where r_d is the maximum client radius in input instance I_d, $OPT(R_i)$ is the total length of paths of the optimal solution on the ith areas divided by j_2 on I_d.*

Proof. Firstly, let r_{R_i} be the radius of client farthest from the originating site in area R_i, since the area farthest from the originating site R_- is $(\epsilon^{j_2} r_d, r_d]$ and the ith$(i \geq 0)$ area is $(\epsilon^{j_2} r_d \epsilon^{(i+1)/\epsilon}, \epsilon^{j_2} r_d \epsilon^{i/\epsilon}]$, $r_{R_-} \leq r_d$ and $r_{R_i} < \epsilon^{i/\epsilon} r_d$. Then, let S_{R_i} be the set of paths in S_d serving R_i, obviously, $Length(S_d) = \sum_i Length(S_{R_i})$, and according to lemma17, $Length(S_{R_i}) < OPT(R_i) + (1/\epsilon)^{\mathrm{O}(1/\epsilon)} \cdot r_{R_i}$. Finally, as the sum of areas $\mathrm{O}(n \log n)$ is limited, and $1/(1-\epsilon^{1/\epsilon}) < 1/\epsilon$ when $\epsilon \leq 1/2$, the result is as follows, the lemma is proved.

$$\begin{aligned}\sum\nolimits_i Length(S_{R_i}) &< \sum\nolimits_i OPT(R_i) + (1/\epsilon)^{\mathrm{O}(1/\epsilon)} \sum\nolimits_i r_{R_i} \\ &< \sum\nolimits_i OPT(R_i) + (1/\epsilon)^{\mathrm{O}(1/\epsilon)} (1 + \sum\nolimits_{i\geq 0} \epsilon^{i/\epsilon}) r_d \\ &< \sum\nolimits_i OPT(R_i) + (1/\epsilon)^{\mathrm{O}(1/\epsilon)} \left(1 + \frac{1}{1-\epsilon^{1/\epsilon}}\right) r_d \\ &< \sum\nolimits_i OPT(R_i) + (1/\epsilon)^{\mathrm{O}(1/\epsilon)} \cdot r_d \end{aligned} \tag{41}$$